0 6 1 0 0 4

# fixtures/precheck_slicer/ — T[1 1] (SpecKit-"fast-dev-cycles", plan T-C )

Fixtures proving `constitution/scripts/fastcycle/review/slicer.py`, `constitution/scripts/fastcycle/review/precheck_pack.sh`, and the (today absent) `gate` subcommand of the ALREADY-LANDED `constitution/scripts/fastcycle/review/review_record.py` (T034/T018) will correctly implement `contracts/review-batch-and-precheck.md`'s RB-001.. RB-006 clauses, against the 4 cases named verbatim in `tasks.md` T061:

1. a review request without a pack is accepted today (baseline);
2. golden: a planted lint error appears in the pack, not in the review;
3. golden-bad: a review record with a non-pinned tier or effort is refused;
4. negative control: an unrelated change is not batched with the others.

## Tools under test (mostly not yet implemented — T-C /T , a later task)

| Tool | State today | CLI (contract `review-batch-and-precheck.md` "Invocations", verbatim) |
|---|---|---|
| `$FC/review/slicer.py` | **absent** (confirmed by this RED test's Section A) | `slicer.py --config <cfg> --changes <sha-range\|list> --slice-limit <lines> --out <batch.json>` |
| `$FC/review/precheck_pack.sh` | **absent** | `precheck_pack.sh --config <cfg> --batch <batch.json> --clean-checkout <dir> --out <precheck.json>` |
| `$FC/review/review_record.py gate` | **absent** (the `record` / `backfill` subcommands ARE landed, T018/T034; `gate` is not registered — confirmed empirically, `python3 review_record.py gate ...` → real argparse "invalid choice: 'gate'", exit 2) | `review_record.py gate --change <sha> --records <dir>` |
| `$FC/review/review_record.py record` | **landed** (T034), used here for real to ground fixture assumptions, never as a stub | `review_record.py record --batch B --round N --verdict-file V --tier T --effort E --out O` |
| `$FC/review/review_record.py backfill` | **landed** (T034), used here for real — this is the ONLY currently-real path that can place a non-pinned-tier record on disk, since `record` refuses one at write time | `review_record.py backfill --input SPEC --out O` |

## `--changes <list>` wire format (UNCONFIRMED by the contract beyond "sha-range|list" — DEFINED here for this RED test's own GREEN-branch, binding-if-adopted on T-C 's implementer, per house precedent set by `fixtures/verdict_cache/README.md` / `fixtures/io_trace/README.md`)

A comma-separated list of paths to per-change descriptor JSON files, each `{"change_id": "sha256:<64-hex>", "path": "<changed path>", "diff": "<sibling diff file>", "lines_changed": <int>}`. `change_id` is a real `sha256:` `ContentAddress` (data-model.md §0) computed over the sibling diff file's real bytes — never a fabricated hex string.

## Case — `case1_no_pack/` (baseline needle, not an absent-tool claim)

`batch.json` + `verdict.json` for a single clean change ( `GO` , zero findings, designated tier/effort). This RED test invokes the REAL, already- landed `review_record.py record` against this fixture with **no** `--precheck` flag and **no** sibling `precheck.json` file present, and asserts the REAL result: exit 0, output JSON `precheck_used: false` . This proves, with a genuine tool execution (never assumed), that a review request with no machine pre-check pack is accepted TODAY — the "before" state `precheck_pack.sh` /a future dispatch guard will change. A sibling control needle confirms `scripts/hooks/review_dispatch_guard.sh` (the NEW hook plan.md line 236 names for T-C11) does not exist anywhere in the repo yet — there is currently no enforcement mechanism at all.

## Case — `case2_lint_in_pack/`

Two real, planted defects, each independently self-validated by this RED test's Section B against a REAL live tool run (§11.4.107(10)/§11.4.273 — "the path is part of the instrument") BEFORE any claim is made about the absent `precheck_pack.sh`:

- `broken_syntax_gate.sh` — a genuine unclosed `if` (real parse error, `sh -n` / `bash -n` on this exact file reports "line 11: syntax error: unexpected end of file").
- `unquoted_var_gate.sh` — a genuine shellcheck SC2086 (real, unquoted variable expansion at `cat $STAGE_DIR/file.txt`, `shellcheck -s sh` reports `SC2086` at line 13 column 5 — the same classifier rule literal T018's `review_record` fixtures already use for the "mechanical" finding class).

`batch.json` batches both files as one change set (their real sha256 as change ids). `expected_precheck.json` documents the GREEN-branch shape (`all_pass: false`, a `parse` FAIL entry and a `shellcheck` FAIL entry, each citing the exact real evidence string reproduced above) — this RED test does not compare byte-for-byte against it (schema/ordering is T-C11's implementer's decision); it asserts only the two load-bearing facts once `precheck_pack.sh` is real.

`clean_review_verdict.json` (`GO`, **zero** findings) is fed to the REAL, already-landed `review_record.py record` for the SAME batch — a real invocation, expected real exit 0 — and this RED test parses the real output JSON to confirm its `findings` array is empty, i.e. the recorded review carries no independent finding about the lint. Together these prove "the error [is] in the pack, not in the review" with two real artifacts: one hand-documented-and-self-validated (the absent tool's future pack), one genuinely produced by a real, already-landed tool (the review record).

## Case — `case3_wrong_tier_gate/`

**Distinct from, and does not duplicate,** T018's `fixtures/review_record/rb_bad_wrong_tier` / `rb_bad_low_effort` (which already prove `record` itself refuses a non-pinned tier/effort AT WRITE TIME — that check is already landed and already GREEN). This fixture exercises the genuinely different, defense-in-

depth question RB-004/RB-006 also demand: once a non-pinned-tier record somehow exists ON DISK, does the absent `gate` subcommand still correctly refuse to count it as coverage?

`backfill_wrong_tier.json` / `backfill_low_effort.json` are `--input` specs for the REAL, already-landed `review_record.py backfill` subcommand — empirically confirmed (before this file was written) that `backfill`, unlike `record`, does **not** validate `model_tier` / `effort` (it exists precisely to back-fill historical, pre-tool review rounds, data-model.md §10.1) — so `backfill` is the only currently-real path that can place a `model_tier: "sonnet"` or `effort: "high"` `ReviewVerdictRecord` on disk. This RED test runs `backfill` for real against both specs, then invokes the absent `gate --change <sha> --records <dir>` against each resulting real record file. `change_under_test.json` names the single change (real sha256 of `case3_change.diff`) both bad records nominally[4] cover; `expected_gate.json` documents the required outcome (exit 1, the change named in stdout) for both.

## Case — `case4_unrelated_not_batched/`

`config.yaml` is this RED test's own fixture-level definition of the `--config`-driven "shared logic group" relation RB-001/DEC-06 name (UNCONFIRMED by any schema beyond "config-defined relation" — binding-if-adopted). It maps `widget_a.sh` / `widget_a_helper.sh` to logic group `widget_a` and `widget_c_unrelated.sh` to a DIFFERENT logic group `widget_c`. `change_widget_a_1.json` / `change_widget_a_2.json` (real sha256 change ids over their sibling `.diff` files) are genuinely related (same logic group) and MUST land in one batch; `change_widget_c_unrelated.json` is genuinely unrelated and MUST NOT be batched with them (RB-001). This RED test invokes the absent `slicer.py --config config.yaml --changes change_widget_a_1.json,change_widget_a_2.json,change_widget_c_unrelated.json --slice-limit 400 --out batch.json`. `expected_batch.json` documents the two load-bearing RB-001 facts (never a byte-for-byte comparison target): the two related changes share a batch, the unrelated change does not, and the union of every produced batch's changes equals all 3 inputs (conservation).

## What this RED test does NOT cover

- The full DEC-33 pack (10 checks total — RB-002 lists `parse`, `shellcheck`, `affected-gates`, `touched-gate-mutations`, `secret-scan`, `doc-sync`, `closure-evidence-class`, `sibling-search`, `blast-radius`, `already-fixed-markers`). This fixture set exercises exactly the two checks (`parse`, `shellcheck`) needed for T061's own case-2 wording ("a planted lint error") — the remaining eight are T-C11's implementer's own scope, not silently claimed tested here.
- `review_record.py report` (rounds-per-change / first-round-GO-rate metrics) and the full 3-change/round-2-GO `rb_good_batched` scenario from the contract's RED fixtures table — out of T061's own 4 named cases, handed to whichever later task drives `report` GREEN.
- `rb_bad_edited_after_go` (content-address staleness after amendment) and `rb_batch_conservation`'s own dedicated fixture — RB-001's conservation invariant is covered here via case 4's `expected_batch.json` instead of a separate scenario, to stay inside T061's own 4-named-case scope.
- Building/wiring the paired-mutation corpus entry itself ("skip the tier check → the golden-bad FAILs") — that is T063's separate, later task; this RED test's own case-3 assertions are structured (real records on disk, real change-id argument, real invocation) so a future skip-the-check mutation on `gate` would genuinely flip them to FAIL, not so it is performed here.